

Node.js Backend MQTT Subscriber

Add this to your existing DigiSaathi Node.js backend:

```
javascript
```

```
// mqtt-handler.js
```

```
// Add this file to your existing Node.js backend
```

```
// Run: npm install mqtt firebase-admin
```

```
const mqtt = require('mqtt');
```

```
const admin = require('firebase-admin');
```

```
// Connect to HiveMQ Cloud
```

```
const mqttClient = mqtt.connect('mqtt://YOUR_CLUSTER.s1.eu.hivemq.cloud:8883', {  
  username: 'YOUR_HIVEMQ_USERNAME',  
  password: 'YOUR_HIVEMQ_PASSWORD',  
  clientId: 'digiSaathi_server_' + Math.random().toString(16).substr(2, 8)  
});
```

```
mqttClient.on('connect', () => {  
  console.log('Backend MQTT connected');  
  // Subscribe to ALL senior command topics  
  mqttClient.subscribe('digiSaathi/+command');  
});
```

```
mqttClient.on('message', async (topic, messageBuffer) => {  
  const topicParts = topic.split('/');  
  const seniorId = topicParts[1];
```

```
const message = JSON.parse(messageBuffer.toString());
```

```
console.log(`Command from ${seniorId}:`, message);
```

```
// 1. Write to Firebase Realtime Database
```

```
await admin.database()
```

```
.ref(`/remote_commands/${seniorId}`)
```

```
.set({
```

```
  ...message,
```

```
  receivedAt: Date.now(),
```

```
  status: 'pending'
```

```
});
```

```
// 2. Send FCM push notification to senior's app
```

```
await sendFCMToSenior(seniorId, message);
```

```
// 3. If SOS, also notify family and guards
```

```
if (message.type === 'sos') {
```

```
  await handleSOS(seniorId);
```

```
}
```

```
// 4. Send ACK back to device
```

```
const ackTopic = `digisaathi/${seniorId}/ack`;
```

```
mqttClient.publish(ackTopic, JSON.stringify({
```

```
  status: 'received',
```

```
  timestamp: Date.now()
```

```
    ));  
  });
```

```
async function sendFCMToSenior(seniorId, command) {  
  // Get senior's FCM token from Firestore  
  const seniorDoc = await admin.firestore()  
    .collection('seniors')  
    .doc(seniorId)  
    .get();  
  
  if (!seniorDoc.exists) return;  
  
  const senior = seniorDoc.data();  
  const fcmToken = senior.fcmToken;  
  
  if (!fcmToken) return;  
  
  // Map type to screen route  
  const routeMap = {  
    'sos': { screen: 'SOS', params: {} },  
    'society': { screen: 'Society', params: { optionIndex: command.optionIndex } },  
    'needs': { screen: 'Needs', params: { optionIndex: command.optionIndex } }  
  };  
  
  const route = routeMap[command.type] || { screen: 'Home', params: {} };
```

```

await admin.messaging().send({
  token: fcmToken,
  notification: {
    title: command.type === 'sos'
      ? '🚨 SOS Alert Activated'
      : `DigiSaathi Remote: ${command.label}`,
    body: command.type === 'sos'
      ? 'Emergency button pressed on your remote'
      : `Navigating to ${command.label}`,
  },
  data: {
    type: command.type,
    optionIndex: String(command.optionIndex),
    label: command.label,
    screen: route.screen,
    fromRemote: 'true'
  },
  android: {
    priority: command.type === 'sos' ? 'high' : 'normal'
  }
});
}

```

```

async function handleSOS(seniorId) {
  const seniorDoc = await admin.firestore()
    .collection('seniors')

```

```
.doc(seniorId)
```

```
.get();
```

```
if (!seniorDoc.exists) return;
```

```
const senior = seniorDoc.data();
```

```
const familyTokens = senior.familyFCMTokens || [];
```

```
// Notify all family members
```

```
for (const token of familyTokens) {
```

```
  await admin.messaging().send({
```

```
    token,
```

```
    notification: {
```

```
      title: '🚨 EMERGENCY — ' + senior.name,
```

```
      body: `${senior.name} has pressed the SOS button. Address: ${senior.address}`
```

```
    },
```

```
    android: { priority: 'high' }
```

```
  });
```

```
}
```

```
// Log SOS event in Firestore
```

```
await admin.firestore()
```

```
.collection('sos_events')
```

```
.add({
```

```
  seniorId,
```

```
  seniorName: senior.name,
```

```
    address:    senior.address,  
    timestamp:  admin.firestore.FieldValue.serverTimestamp(),  
    status:     'active',  
    familyNotified: familyTokens.length  
  });  
}
```

```
module.exports = { mqttClient };
```

Then in your main server.js or index.js, add:

```
javascript
```

```
require('./mqtt-handler');
```

Agent Prompt — App Side Changes

Give this entire block to your AI coding agent:

You are working on DigiSaathi — a React Native (Android-first) digital literacy app for senior citizens built at RSCOE under the ACM student chapter. Stack: React Native frontend, Node.js backend, Firebase Firestore for data, Firebase Realtime Database for real-time sync, FCM for push notifications, Google Speech-to-Text for voice, OTP authentication.

A physical IoT remote (ESP32-based) has been added to the system. The remote communicates via MQTT to a HiveMQ Cloud broker. The Node.js backend subscribes to MQTT, writes commands to Firebase Realtime Database at `/remote_commands/{seniorId}`, and sends FCM push notifications to the app.

The FCM notification data payload has these fields:

```
{  
  "type": "sos" | "society" | "needs",  
  "optionIndex": "0" to "9" (string, parse to int),  
  "label": "option name string",  
  "screen": "SOS" | "Society" | "Needs",  
  "fromRemote": "true"  
}
```

The Firebase Realtime Database node written is:

```
/remote_commands/{seniorId} → {
```

```
type, optionIndex, label, deviceId, timestamp,  
receivedAt, status: "pending"  
}
```

Firestore data model for reference:

```
/seniors/{seniorId} → {  
  name, phone, address, societyId,  
  fcmToken, familyFCMTokens[], familyContacts[],  
  guardContacts[], pairedDevices[]  
}  
  
/societies/{societyId}/complaints/ → collection  
  
/societies/{societyId}/vendors/{category} → vendor info  
  
/orders/{seniorId}/{orderId} → order records
```

YOUR TASKS — implement ALL of the following completely:

TASK 1 — RemoteCommandListener.js (new service file)

Create src/services/RemoteCommandListener.js.

This service must:

- Export a function startRemoteListener(seniorId, navigation)
- Listen to Firebase Realtime Database path
 /remote_commands/{seniorId} using onValue
- Only process commands where status === "pending" AND
 receivedAt is within the last 30 seconds (to ignore stale)

- Immediately set status to "processing" after receiving
- Route based on type field:
 - "sos" → call handleSOSCommand()
 - "society" → navigation.navigate("Society", { optionIndex, fromRemote: true })
 - "needs" → navigation.navigate("Needs", { optionIndex, fromRemote: true })
- After routing set status to "completed" in Realtime Database
- Handle listener cleanup — return unsubscribe function

TASK 2 — Integrate in App.js

In App.js, after user authentication is confirmed (user object is non-null), call:

```
startRemoteListener(user.seniorId, navigationRef)
```

Store the unsubscribe function and call it on logout/cleanup.

Use a navigationRef created with createNavigationContainerRef() so navigation works from outside component tree.

TASK 3 — SOS Modal (SOSAlert.js)

Create src/components/SOSAlert.js — a full-screen modal overlay.

Must:

- Accept visible (bool), seniorData (object), onDismiss (func) as props
- Show large red banner "  SOS ACTIVATED"
- Show senior name and address in large text (min 20sp)
- Show timestamp formatted as "DD MMM YYYY, HH:MM"
- Show animated pulsing red circle to indicate active alert
- Have a "Mark Safe ✓" button (large, green, full width) that calls onDismiss and sends FCM to family saying senior is safe

- Auto-dismiss after 5 minutes — show countdown timer in UI
- Background color: #1a0000, text color: white

In handleSOSCommand() inside RemoteCommandListener.js:

- Set global state to show SOSAlert modal
- Fetch senior data from Firestore /seniors/{seniorId}
- Pass data to modal

TASK 4 — Society Screen (SocietyScreen.js)

Create src/screens/SocietyScreen.js if it does not exist, or modify existing society/complaints screen.

Must:

- Accept route.params: { optionIndex, fromRemote }
- Show top banner if fromRemote is true:
 - "  Request from DigiSaathi Remote" in blue banner
- Display all 10 society options as large tappable cards:
 - 0=Water Issue, 1=Electricity, 2=Cleaning, 3=Lift Problem,
 - 4=Security, 5=Noise Issue, 6=Gate Access, 7=Maintenance,
 - 8=Society Info, 9=First Aid

Each card: icon + label, min height 80dp, min font 18sp
- Pre-select the card matching optionIndex on mount
- Selected card highlighted with blue border
- Show text input "Describe your issue (optional)" below
- Large "Submit Request" button (full width, 56dp height)
- On submit: write to Firestore at
 - /societies/{societyId}/complaints/{autoId} with fields:
 - { seniorId, seniorName, category, optionIndex, description,

timestamp: serverTimestamp(), status: "open" }

- Send FCM to society manager after Firestore write
- Show success screen: "Request Submitted ✓" with complaint ID
- Navigate back to home after 3 seconds

TASK 5 — Daily Needs Screen (NeedsScreen.js)

Create src/screens/NeedsScreen.js if it does not exist.

Must:

- Accept route.params: { optionIndex, fromRemote }
- Show top banner if fromRemote is true (same as society screen)
- Display 10 category cards matching the needs options list:
0=Vegetables & Fruits, 1=Grocery, 2=Medicines, 3=Milk & Dairy,
4=Tiffin/Food, 5=Water Can, 6=Newspaper, 7=Laundry,
8=House Help, 9=Courier Help
- Pre-select card matching optionIndex on mount
- On category selection: fetch vendor data from Firestore at
/societies/{societyId}/vendors/{categoryName}
- Show vendor name, available items with prices, quantity
selectors (+ / - buttons, large touch targets min 48dp)
- Show "Saved Address" fetched from senior's Firestore profile
- Large "Place Order" button
- On order: write to /orders/{seniorId}/{autoId}:
{ seniorId, vendorId, category, items[], totalAmount,
deliveryAddress, timestamp: serverTimestamp(), status: "placed" }
- Send FCM notification to vendor's device
- Show order confirmation with order number

TASK 6 — Device Pairing Screen (PairDeviceScreen.js)

Create src/screens/PairDeviceScreen.js.

Must:

- Accessible from Settings screen
- Import QRCode from react-native-qrcode-svg
- Display QR code generated from seniorId string
- Show instructions: "Show this QR code during device setup"
- Show paired device status:
 - Connected = green dot + "Remote connected" if last command was within 5 minutes
 - Disconnected = grey dot + "Remote not detected"(check /remote_commands/{seniorId}/receivedAt)
- Show last command received: type + label + formatted time
- Button "Unpair Device" — removes deviceId from /seniors/{seniorId}/pairedDevices array

TASK 7 — Background FCM Handler

In the FCM background message handler (wherever it is currently set up, or create it):

When a notification arrives with data.fromRemote === "true":

- Parse type and optionIndex from notification data
- Show a local notification with title and body from the FCM
- When user taps notification, deep link to correct screen with pre-selection params already set:

```
type=society → navigate("Society", { optionIndex: parseInt(data.optionIndex), fromRemote: true })
```

```
type=needs → navigate("Needs", { optionIndex: parseInt(data.optionIndex), fromRemote: true })
```

```
type=sos → show SOSAlert modal
```

TASK 8 — Firebase Realtime Database Rules

Update database.rules.json:

```
{
  "rules": {
    "remote_commands": {
      "$seniorId": {
        ".read": "auth != null && auth.uid === $seniorId",
        ".write": "auth != null && (auth.uid === $seniorId ||
          root.child('seniors').child($seniorId)
            .child('pairedDevices')
            .child(auth.token.deviceId).exists())"
      }
    }
  }
}
```

GENERAL REQUIREMENTS FOR ALL SCREENS:

- Minimum font size 18sp throughout, touch targets min 48dp
- Colors: primary blue #1565C0, SOS red #B71C1C, success green #2E7D32, background white #FFFFFF
- All screens must support Hindi and Marathi via i18n

(set up react-i18next if not present)

- Do not modify or break existing app navigation
- Add new screens to the existing navigation stack
- All new screens must have a back button in header
- Show loading spinners during all async Firestore operations
- Handle network errors gracefully with retry button
- Test by manually writing this to Realtime Database:

```
/remote_commands/senior_001 = {  
  type: "society", optionIndex: 2, label: "Cleaning",  
  status: "pending", receivedAt: Date.now()  
}
```

Confirm app navigates to Society screen with option 2 selected.